



ESCUELA DE
INGENIERÍA EN CIENCIAS Y SISTEMAS
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



Día, Fecha:	Viernes, XX-XX-XXXX
Hora de inicio:	19:00

Inteligencia Artificial 1 [A]

Robin Omar Buezo Díaz

INTELIGENCIA ARTIFICIAL 1

Clase 3

Lógica e Inferencia

Escuela de Ingenieria de Ciencias Y Sistemas
Facultad de Ingenieria
Universidad de San Carlos de Guatemala

Elaborado por: Robin Omar Buezo Díaz

Agenda

Recordatorios

Temas

- Negación y Cortes
- Ciclos y Recursión

Recursos

Link del Video



Actividad

- UEDI

Elaborado por: Robin Omar Buezo Díaz

COMPETENCIA(S) Que desarrollaremos

Desarrolla hechos, reglas, expresiones y predicados recursivos mediante el uso de cláusulas, ciclos, listas, unificación y cortes en Tau Prolog para modelar bases de conocimiento y resolver problemas de inferencia lógica.

Cortes

El corte, representado por el símbolo `!`, es un predicado incorporado en Prolog que sirve para controlar el backtracking (retroceso) durante la ejecución del programa. Su función principal es "podar" o eliminar alternativas de búsqueda, permitiendo al programador restringir el comportamiento de Prolog al explorar soluciones.

```
P :- A1, ..., Ai, !, Ai+1, ..., An.  
P :- B1, ..., Bm.  
P :- C1, ..., Cp.
```

```
P :- A1, ..., Ai, !, Ai+1, ..., An.  
P :- B1, ..., Bm.  
P :- C1, ..., Cp.
```

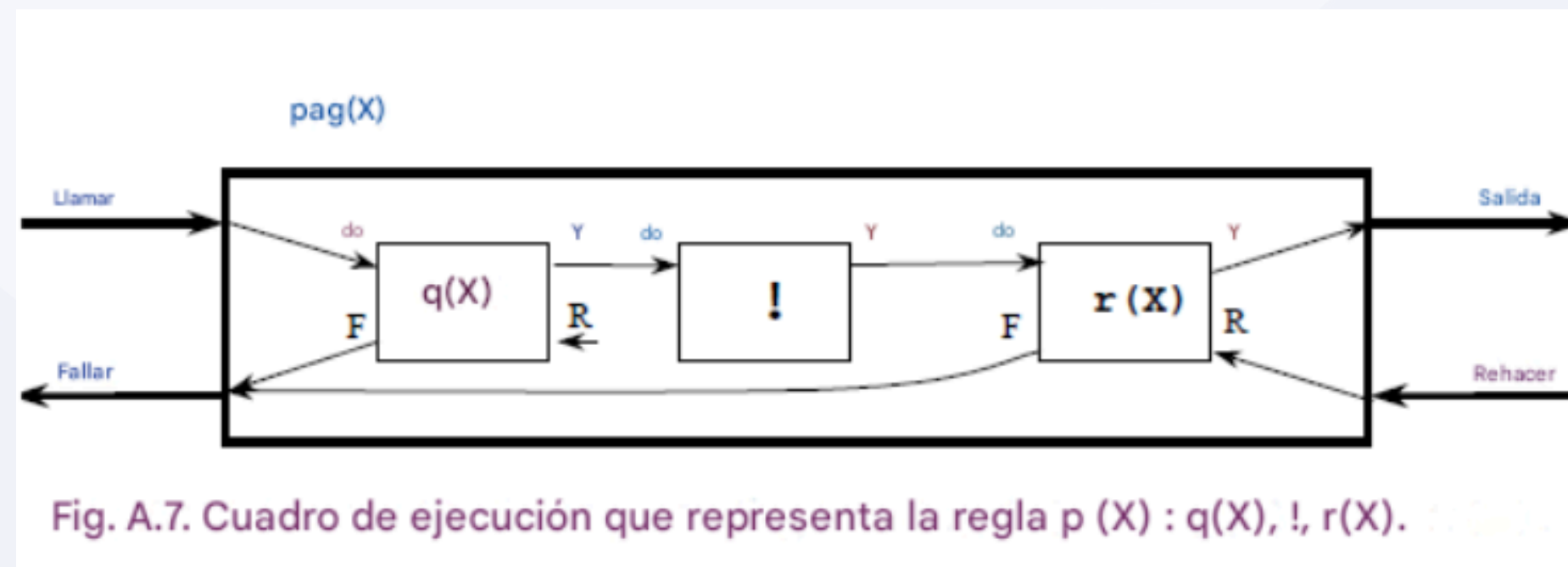
El corte es una herramienta poderosa para optimizar programas Prolog, pero requiere un uso cuidadoso para evitar la eliminación accidental de soluciones. Su función principal es controlar el backtracking y mejorar la eficiencia.

Elaborado por: Robin Omar Buezo Díaz

¿Qué hace exactamente el corte?

Cuando Prolog encuentra un ! (corte) en una cláusula:

- Se compromete con todas las decisiones tomadas antes del corte dentro de esa cláusula.
- Si luego la ejecución falla, Prolog no volverá a considerar otras cláusulas alternativas del mismo predicado ni otras soluciones anteriores a ese punto.
- El corte no produce efectos por sí mismo, pero afecta el flujo de control.



Tipos de Corte

Corte Verde

- Se utiliza solo para eficiencia.
- No cambia la lógica del programa, solo evita búsquedas innecesarias.
- Correcto y seguro.

max(X, Y, X) :- X >= Y, !.

max(_, Y, Y).

- Si $X \geq Y$ es cierto, se elige la primera cláusula, y el corte evita evaluar la segunda.
- El corte aquí es verde, porque no cambia la lógica, solo mejora el rendimiento.

Corte Rojo

- Se utiliza para forzar decisiones lógicas.
- Afecta el significado del programa (puede descartar soluciones válidas).
- Debe usarse con precaución.

clasificacion(X, positivo) :- X > 0, !.

clasificacion(X, negativo) :- X < 0, !.

clasificacion(0, neutro).

- Aquí el corte impide que se evalúe la tercera cláusula incluso si $X = 0$.
- Este es un corte rojo, porque modifica la lógica del programa.

- *El corte es útil pero puede dificultar la depuración y el razonamiento lógico.*
- *En programas declarativos, se recomienda usar el corte solo cuando sea necesario y entendido.*
- *En muchos casos, hay alternativas más limpias y declarativas al uso del corte.*

Elaborado por: Robin Omar Buezo Díaz

Negación

En Prolog, la negación se maneja de forma no clásica debido a su base en la lógica de programación basada en pruebas. No se representa información negativa directamente como en otros lenguajes declarativos o lógicos, sino que se utiliza una técnica llamada:

Negación como falla (Negation as Failure, NAF)

¿Cómo funciona?

En Prolog, decir que algo no es cierto se interpreta como "no se puede probar que sea cierto".

Esto significa que:

- Si una consulta G tiene éxito, entonces $+ G$ (no G) falla.
- Si una consulta G falla, entonces $+ G$ (no G) tiene éxito.

Esto se representa con el operador:

- $\backslash + G \rightarrow \text{"no } G\text{"}$
- También puede verse como $\text{not}(G)$ en versiones antiguas o algunos entornos.

Ejemplo

```
hombre(juan).  
hombre(pedro).
```

```
mujer(maria).
```

```
% Consultas
```

```
?- \+ hombre(maria).
```

```
true. % Porque no puede probar que maria sea hombre
```

```
?- \+ mujer(juan).
```

```
true. % Porque no puede probar que juan sea mujer
```

```
?- \+ mujer(maria).
```

```
false. % Porque puede probar que maria sí es mujer
```

¿Cómo se implementa internamente?

Internamente, la negación como falla puede modelarse usando un corte (!) seguido de una regla que siempre falla (fail), como esta:

```
\+(P) :- P, !, fail.  
\+(P) :- true.
```

Esto se interpreta así:

- Si G es cierto (G tiene éxito), el corte ! impide cualquier otra alternativa y fail fuerza el fallo.
- Si G falla, entonces la segunda cláusula (not(_)) tiene éxito.

Este patrón evita que Prolog siga buscando soluciones alternativas y define claramente el comportamiento del not.

Consideraciones importantes

- **Incompleto:** La negación como falla no es lógica pura; puede dar resultados incorrectos si hay variables no instanciadas.
- **No es lógica clásica:** No se debe asumir que $\neg p(X)$ significa que $\neg(p(X))$ en lógica formal.
- **Cuidado con predicados no cerrados:** Si la base de datos no está completa, las conclusiones pueden ser engañosas.

Ciclos y Recursión

En lenguajes imperativos como Python, Java o C, las estructuras de control como for, while o do-while permiten realizar tareas repetitivas de manera explícita. Sin embargo, Prolog, como lenguaje declarativo, no posee estructuras de iteración tradicionales. En su lugar, la recursión es el principal mecanismo para procesar estructuras repetitivas o listas, realizar acumulaciones, y explorar soluciones en árboles de búsqueda.

Tipos de Recursión

Recursión Directa

Una función se llama a sí misma en su definición.

factorial(0, 1). % Caso base: factorial de 0 es 1

factorial(N, F) :-

N > 0,

N1 is N - 1,

factorial(N1, F1), % Llamada recursiva

*F is N * F1.*

Aquí, la llamada recursiva ocurre antes de la multiplicación. Esto requiere que el intérprete mantenga cada llamada en la pila, lo que puede provocar desbordamientos para valores grandes de N.

Recursión de Cola

Optimiza el uso de memoria, ya que la última llamada en la función es la recursión.

factorial_tail(N, F) :- factorial_helper(N, 1, F).

factorial_helper(0, Acc, Acc). % Caso base

factorial_helper(N, Acc, F) :-

N > 0,

N1 is N - 1,

*Acc1 is Acc * N,*

factorial_helper(N1, Acc1, F).

Acc actúa como un acumulador para llevar el resultado parcial.

Ventajas de la Recursión en Prolog

**Se alinea con el paradigma
declarativo**

**Permite expresar soluciones
elegantes y concisas**

**Facilita el procesamiento de
listas y estructuras anidadas**

¿Cuándo usar recursión de cola?

Al procesar grandes volúmenes de datos o listas extensas.

Para evitar el uso excesivo de pila.

Siempre que sea posible, ya que mejora el rendimiento y eficiencia.

Conceptos Clave Aprendidos

- **Corte en Prolog:** El corte (!) controla el retroceso en la ejecución. El corte verde mejora la eficiencia sin cambiar la lógica. El corte rojo modifica la lógica del programa y debe usarse con precaución.
- **Negación como Falla:** Prolog usa \+ para negar: algo es falso si no se puede probar como verdadero. No es lógica clásica y tiene limitaciones con variables no instanciadas.
- **Recursión:** Prolog utiliza recursión en lugar de bucles. La recursión directa es más simple, pero la de cola es más eficiente y evita problemas de memoria.
- **Paradigma Declarativo y Simulación:** Prolog describe lo que se quiere lograr, no cómo. Es útil en IA y simulaciones por su enfoque lógico y expresivo.

Valores y Actitudes

- **Responsabilidad Ética:** Aplicar la lógica con conciencia del impacto tecnológico.
- **Razonamiento Crítico:** Pensar con lógica y coherencia para resolver problemas.
- **Precaución Técnica:** Usar herramientas como el corte con responsabilidad.
- **Curiosidad y Aprendizaje:** Explorar nuevos lenguajes y herramientas con iniciativa.
- **Creatividad Lógica:** Diseñar soluciones eficientes con reglas y recursión.
- **Trabajo en Equipo:** Colaborar activamente en entornos multidisciplinarios.

Referencias

- Russell, S., & Norvig, P. (2016). Artificial Intelligence: A Modern Approach (3rd ed.). Pearson.
- Clocksin, W. F., & Mellish, C. S. (2003). Programming in Prolog: Using the ISO Standard (5^a ed.). Springer-Verlag.

¡GRACIAS POR su



DUDAS

Recuerda que tenemos nuestro foro semanal donde puedes consultar cualquier duda que te surja en la semana